

TERRAFORM MASTER INTERVIEW HANDBOOK

From Zero to Enterprise DevOps | Comprehensive Q&A;,, Architecture, Scenarios & Spoken English Answers

Candidate: Priyanshu Jaiswal	Focus: Full Stack Java & Cloud DevOps
Target Companies: Deloitte, EY, HCLTech, Product Firms	Edition: 2026 Enterprise Edition

SECTION 1: CORE IAC & TERRAFORM FUNDAMENTALS

QUESTION 1

What is Infrastructure as Code (IaC) and what core problems does it solve?

Context & Why it's asked: Checks your understanding of why modern cloud engineering moved away from manual AWS Web Console clicking.

Detailed Technical Answer: Infrastructure as Code is the practice of provisioning and managing cloud infrastructure using machine-readable configuration definition files rather than manual physical hardware configuration or interactive web consoles. It eliminates human error, configuration drift, and documentation rot while introducing version control, automated peer reviews, rollbacks, and fast environment replication.

■■ **Spoken English Interview Soundbite:**

"IaC transforms infrastructure from a manual, error-prone clicking process into version-controlled software. By defining infrastructure in code, we achieve automated provisioning, auditability through Git commits, and zero drift across Dev, Stage, and Production environments."

QUESTION 2

How does Terraform's declarative model differ from imperative configuration tools like Ansible or Bash?

Context & Why it's asked: Tests if you understand state convergence and declarative vs imperative paradigms.

Detailed Technical Answer: In an imperative model (like Bash or Ansible playbooks), you define the exact sequence of operational steps: 'create VM, install package, start service'. If the script runs twice, it might fail unless manually made idempotent.

In Terraform's declarative model, you declare the desired end-state ('I want 3 EC2 instances in us-east-1'). Terraform evaluates current state against desired state and automatically calculates the minimum delta required to reach that target.

■■ Spoken English Interview Soundbite:

"The core difference is 'What' versus 'How'. Imperative tools require you to specify every step to execute, whereas Terraform is declarative: we simply specify our desired end-state, and Terraform's graph engine determines the exact API lifecycle steps required to achieve that state."

QUESTION 3

Explain the core Terraform CLI lifecycle: init, plan, apply, and destroy.

Context & Why it's asked: The foundational workflow question asked in every entry-level and mid-level DevOps interview.

Detailed Technical Answer: 1. terraform init: Initializes the working directory, downloads required provider plugins into .terraform/, initializes the remote backend, and resolves modules.

2. terraform plan: Performs an execution dry-run. It refreshes current state from cloud APIs, calculates differences against code, and outputs an execution plan (+ create, ~ update, - destroy) without making real changes.

3. terraform apply: Executes the planned changes against provider APIs to provision or alter infrastructure and updates the state file.

4. terraform destroy: Reverses the dependency graph and safely terminates all resources managed within that state file.

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform destroy
```

■■ Spoken English Interview Soundbite:

"The Terraform workflow follows a disciplined lifecycle: init prepares the workspace and providers; plan performs an idempotent dry-run showing expected deltas; apply provisions the changes to cloud APIs; and destroy tears down the managed infrastructure when no longer needed."

QUESTION 4

What is Idempotency in Terraform and why is it critical for enterprise reliability?

Context & Why it's asked: A critical systems engineering concept asked by technical leads.

Detailed Technical Answer: An operation is idempotent if executing it multiple times produces the exact same result without unintended side effects. In Terraform, if you run 'terraform apply' on an infrastructure that already matches your code, Terraform does nothing: 'No changes. Your infrastructure matches the configuration.' This guarantees that running automated CI/CD pipelines repeatedly will never accidentally spin up duplicate instances or corrupt live data.

■■ **Spoken English Interview Soundbite:**

"Idempotency means running the same configuration multiple times yields the exact same outcome without duplicate resource creation. In production, idempotency prevents accidental resource duplication and ensures our CI/CD pipelines are safe to execute continuously."

SECTION 2: VARIABLES, OUTPUTS & CONFIGURATION PRECEDENCE

QUESTION 5

What is the exact variable definition precedence order in Terraform?

Context & Why it's asked: One of the most frequently asked tricky questions to test practical debugging skills.

Detailed Technical Answer: When a variable is assigned values in multiple places, Terraform resolves them in this strict order (highest priority wins):

1. **-var and -var-file** command-line flags (Highest priority)
2. ***.auto.tfvars** or ***.auto.tfvars.json** files (alphabetical order)
3. **terraform.tfvars.json** file
4. **terraform.tfvars** file
5. **TF_VAR_variable_name** environment variables
6. **default** value declared inside the variable block (Lowest priority)

■■ **Spoken English Interview Soundbite:**

"Terraform evaluates variable precedence from lowest to highest: default block values are overridden by TF_VAR environment variables, which are overridden by terraform.tfvars, auto.tfvars, and finally the command-line -var flag, which always takes highest precedence."

QUESTION 6

How do you implement input variable validation to prevent invalid infrastructure provisioning?

Context & Why it's asked: Distinguishes senior engineers who write defensive code from juniors who accept any input.

Detailed Technical Answer: Terraform allows a **validation {}** block inside variables. It evaluates a boolean condition and returns an error message before making cloud API calls. For instance, restricting instance sizes to free-tier or approved company sizes.

```
variable "instance_type" {
  type      = string
  default   = "t3.micro"
  validation {
    condition     = contains(["t3.micro", "t3.small"], var.instance_type)
    error_message = "Only t3.micro or t3.small instances are permitted."
  }
}
```

■■ Spoken English Interview Soundbite:

"We implement custom validation rules within our variable definitions using the validation block. This ensures parameters conform to organizational compliance—like allowed instance types or naming conventions—failing fast at the plan stage before calling AWS APIs."

SECTION 3: STATE MANAGEMENT, REMOTE BACKENDS & LOCKING

QUESTION 7

What is the Terraform State file and why is storing it in Git an anti-pattern?

Context & Why it's asked: Security and concurrency question asked by Deloitte, EY, and enterprise firms.

Detailed Technical Answer: The state file (terraform.tfstate) is a JSON document mapping your HCL code to physical cloud IDs and metadata. Committing state to Git is dangerous because:

1. **Plain-text Secrets:** Database passwords, private keys, and sensitive tokens are written in plain text in the state file.
2. **Concurrency Collisions:** If two developers run apply simultaneously, Git merge conflicts can corrupt the state.
3. **Lack of State Locking:** Git does not provide distributed locks during execution.

■■ Spoken English Interview Soundbite:

"The state file must never be committed to source control for two critical reasons: security and concurrency. State files store sensitive credentials in plain text, and Git lacks the distributed locking mechanism required to prevent concurrent write corruptions across team members."

QUESTION 8

How do AWS S3 and DynamoDB work together for production Remote State management?

Context & Why it's asked: Standard architecture question for any AWS DevOps engineer.

Detailed Technical Answer: In production, we configure an S3 remote backend. S3 stores the state file centrally with AES-256 encryption at rest, HTTPS in transit, and bucket versioning enabled for rollback recovery. DynamoDB provides **State Locking** via a LockID attribute: before any execution, Terraform acquires a lock in DynamoDB; if another engineer runs plan/apply, their process is blocked until the lock is released. This guarantees zero state corruption.

```
terraform {  
  backend "s3" {  
    bucket      = "company-tf-state-prod"  
    key         = "prod/terraform.tfstate"  
    region     = "us-east-1"  
    dynamodb_table = "terraform-state-locks"  
    encrypt     = true  
  }  
}
```

■■ Spoken English Interview Soundbite:

"In enterprise AWS setups, we decouple state storage and concurrency management: S3 acts as the encrypted, versioned remote storage backend, while DynamoDB provides distributed locking via state lock IDs to eliminate race conditions between engineers and CI/CD pipelines."

QUESTION 9

What happens if a Terraform process crashes leaving DynamoDB locked, and how do you recover?

Context & Why it's asked: Real-world incident management question.

Detailed Technical Answer: If a network connection drops or a CI/CD agent gets killed during apply, the DynamoDB lock remains active. Subsequent runs fail with: 'Error acquiring the state lock: Lock Info: ID...'. To recover, verify no active pipeline is executing, note the Lock ID from the error message, and run **terraform force-unlock <LOCK_ID>**.

■■ Spoken English Interview Soundbite:

"When an apply is forcefully interrupted, the DynamoDB lock can remain orphaned. We resolve this by verifying no pipeline is actively writing, and then executing terraform force-unlock with the specific Lock ID printed in the error console."

SECTION 4: STATE CLI OPERATIONS & TERRAFORM IMPORT

QUESTION 10

Explain the critical difference between 'terraform destroy' and 'terraform state rm'.

Context & Why it's asked: A favorite tricky interview question.

Detailed Technical Answer: terraform destroy: Deletes the infrastructure from BOTH the state file AND the physical cloud provider (terminates the EC2 instance, drops the database, stops live traffic).

terraform state rm: Removes the resource tracking ONLY from the state file. It makes ZERO API deletion calls to AWS; the live server or database continues running completely unharmed.

■■ Spoken English Interview Soundbite:

"terraform destroy terminates resources from both the state file and the cloud provider, stopping live traffic. In contrast, terraform state rm purely untracks the resource from Terraform state, leaving the cloud infrastructure running completely untouched in AWS."

QUESTION 11

What is 'terraform import' and how do you bring legacy, manually created cloud resources into code?

Context & Why it's asked: Legacy migration question essential for consulting roles.

Detailed Technical Answer: When cloud resources were created manually via the AWS console, Terraform is unaware of them. To import them:

1. Write an empty resource block in main.tf (e.g. resource 'aws_s3_bucket' 'legacy' {}).
2. Run: **terraform import aws_s3_bucket.legacy <actual-bucket-name>**.
3. Run **terraform state show aws_s3_bucket.legacy** to inspect the imported attributes.
4. Align the main.tf block with those attributes until 'terraform plan' confirms zero changes.

```
# CLI import
terraform import aws_s3_bucket.legacy my-manual-bucket-name

# Modern Terraform 1.5+ import block
import {
  to = aws_s3_bucket.legacy
  id = "my-manual-bucket-name"
}
```

■■ Spoken English Interview Soundbite:

"terraform import allows us to onboard legacy, unmanaged cloud resources without downtime. We define an empty resource skeleton in code, execute terraform import with the resource address and cloud ID, and then backfill the HCL configuration until terraform plan reports zero drift."

QUESTION 12

How do 'terraform state list', 'state show', and 'state mv' assist in refactoring?

Context & Why it's asked: Checks command-line proficiency and refactoring capability.

Detailed Technical Answer: **state list:** Enumerates all resource paths currently tracked.

state show <res>: Displays the complete JSON attribute dump of a single resource.

state mv <src> <dst>: Renames a resource or moves it into a child module without destroying and recreating it in AWS.

■■ Spoken English Interview Soundbite:

"These state inspection commands are critical for Day-2 refactoring: state list gives an active inventory, state show exposes internal attributes, and state mv allows us to rename resources or refactor them into modules with zero cloud downtime."

SECTION 5: TERRAFORM MODULES (ARCHITECTURE & REUSABILITY)

QUESTION 13

What is a Terraform Module? Explain the relationship between Root Module and Child Module.

Context & Why it's asked: Core architectural question.

Detailed Technical Answer: A module is a container for multiple resources that are used together. It enforces the DRY (Don't Repeat Yourself) principle.

Root Module: The main directory where you run terraform init, plan, and apply.

Child Module: Any reusable package/folder called by the root module via a **module {}** block.

■■ Spoken English Interview Soundbite:

"Think of a Terraform module like a class or function in programming: we write the infrastructure logic once inside a child module, and then invoke it from our root module across Dev, Stage, and Prod with parameterized inputs."

QUESTION 14

Can a Child Module access Root Module variables directly? Explain the data flow.

Context & Why it's asked: Tests understanding of scope isolation.

Detailed Technical Answer: NO! Module scopes are strictly isolated. A child module cannot see root module variables. Data flow is explicit:

1. **Inputs (Root to Child):** Root passes arguments into the module block, which the child module receives in its `variables.tf`.
2. **Outputs (Child to Root):** The child exports results in `outputs.tf`, which the root accesses via `module.<name>.<output_name>`.

■■ Spoken English Interview Soundbite:

"Module scopes are strictly encapsulated. A child module cannot read root variables directly. Root explicitly passes values into child variables, and receives return values through module outputs."

QUESTION 15

What is the Module Renaming Trap and how does the 'moved {}' block solve it?

Context & Why it's asked: Senior-level scenario question.

Detailed Technical Answer: If you rename a module in code (e.g. from module 'web' to module 'frontend'), Terraform evaluates it as: destroy module.web (terminating production servers!) and create module.frontend from scratch. To achieve zero-downtime refactoring without manual state commands, Terraform 1.1+ introduced the `moved {}` block. Terraform updates the state address automatically without touching the cloud resource.

```
moved {  
  from = module.web  
  to   = module.frontend  
}
```

■■ Spoken English Interview Soundbite:

"Renaming a module without migration instructions causes Terraform to destroy the old resource and recreate the new one. The moved block instructs Terraform that the resource was simply relocated in code, updating the state pointer with zero infrastructure downtime."

SECTION 6: MULTI-ENVIRONMENT STRATEGIES (WORKSPACES VS DIRECTORIES)

QUESTION 16

What are Terraform Workspaces and how does 'terraform.workspace' function?

Context & Why it's asked: Understanding workspace state isolation.

Detailed Technical Answer: Workspaces allow a single configuration directory to maintain multiple distinct state files (e.g. default, dev, prod). Switching workspaces via **terraform workspace select dev** changes the active state pointer. The built-in **terraform.workspace** variable allows dynamic parameterization (e.g. `instance_type = terraform.workspace == 'prod' ? 't3.large' : 't3.micro'`).

■■ **Spoken English Interview Soundbite:**

"Workspaces provide state isolation within a single codebase. By switching workspaces, Terraform isolates state files while letting us dynamically interpolate the terraform.workspace string into resource tags and sizes."

QUESTION 17

Compare Workspaces vs Directory Isolation with tfvars for Dev/Stage/Prod. Which is preferred?

Context & Why it's asked: Deloitte and enterprise cloud architecture favorite.

Detailed Technical Answer: Workspaces share the exact same backend credentials and codebase. If an engineer runs destroy in the wrong workspace or a bug exists in code, the blast radius is large.

Directory Isolation (e.g. environments/dev and environments/prod) maintains separate folders pointing to separate AWS accounts and isolated S3 state buckets. **In enterprise production, Directory Isolation combined with reusable Modules is the gold standard** because it enforces hard security boundaries.

■■ **Spoken English Interview Soundbite:**

"While Workspaces are useful for identical short-lived feature testing, Directory Isolation combined with reusable Modules is the enterprise standard. Directory isolation limits blast radius by binding Dev and Prod to separate AWS accounts and isolated state storage."

SECTION 7: DEPENDENCIES, DIRECTED ACYCLIC GRAPH & CYCLE ERRORS

QUESTION 18

Explain the difference between Implicit Dependencies and Explicit Dependencies ('depends_on').

Context & Why it's asked: Classic Terraform mechanics question.

Detailed Technical Answer: Implicit Dependency: Formed automatically when Resource B references an attribute of Resource A (e.g. `subnet_id = aws_subnet.main.id`). Terraform's DAG engine infers that the subnet must be provisioned before the EC2 instance.

Explicit Dependency (depends_on): Used when two resources have no direct HCL reference, but a real-world runtime dependency exists (e.g. EC2 user_data script downloading a file from S3, or IAM Role policies that must propagate before an instance profile binds).

```
resource "aws_instance" "web" {  
  ami           = "ami-0c7217cdde317cfec"  
  instance_type = "t3.micro"  
  user_data     = "aws s3 cp s3://app-bucket/app.jar ."  
  depends_on   = [aws_s3_bucket.app_bucket]  
}
```

■■ Spoken English Interview Soundbite:

"Implicit dependencies are inferred automatically when one resource references another's attributes. Explicit dependencies, defined via the `depends_on` meta-argument, are required when resources are decoupled in code but have a strict ordering requirement at runtime."

QUESTION 19

What causes a Circular Dependency ('Cycle Error') in Terraform and how do you resolve it?

Context & Why it's asked: Tests your understanding of Terraform's internal DAG engine.

Detailed Technical Answer: A Cycle Error occurs when Resource A depends on Resource B, while Resource B simultaneously depends on Resource A. Terraform builds a Directed Acyclic Graph (DAG); cycles create an infinite loop. Resolution: break the cycle by decoupling the shared attribute into a third standalone resource (e.g., using a standalone `'aws_security_group_rule'` instead of inline rules inside `'aws_security_group'`).

■■ Spoken English Interview Soundbite:

"A Cycle Error indicates an infinite loop where two resources depend on each other. We resolve this by decoupling the circular attributes into standalone helper resources, such as splitting inline security group rules into independent rule resources."

SECTION 8: RESOURCE LIFECYCLE RULES (PRODUCTION ZERO-DOWNTIME)

QUESTION 20

What is the default lifecycle of a Terraform resource and why is it dangerous in production?

Context & Why it's asked: The fundamental motivation behind lifecycle rules.

Detailed Technical Answer: The default lifecycle actions are: Create (+), Update in-place (~), Destroy & Recreate (-/+), and Destroy (-).

The danger lies in **Destroy & Recreate (-/+)**: by default, Terraform terminates the old resource first, and only then creates the replacement. If updating an AMI, the server is terminated before the new one boots, resulting in 5 to 10 minutes of application downtime.

■■ **Spoken English Interview Soundbite:**

"By default, when a property change forces recreation, Terraform destroys the existing resource before provisioning the replacement. In production, this causes downtime. We override this using the lifecycle block."

QUESTION 21

How does 'create_before_destroy = true' achieve Zero-Downtime rolling deployments?

Context & Why it's asked: High-frequency interview question.

Detailed Technical Answer: It reverses the replacement order: Terraform provisions the replacement resource first, confirms it is healthy, updates downstream load balancer targets, and only then terminates the old resource. **Caveat:** Resources with unique naming constraints will collide; use **name_prefix** instead of static names.

```
resource "aws_instance" "web" {  
  ami           = "ami-0c7217cdde317cfec"  
  instance_type = "t3.micro"  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

■■ **Spoken English Interview Soundbite:**

"create_before_destroy flips Terraform's default sequence: it stands up the new resource first, verifies readiness, and then tears down the old one, eliminating availability gaps during server or certificate replacements."

QUESTION 22

Explain 'prevent_destroy = true'. What happens if you run 'terraform destroy'?

Context & Why it's asked: Mission-critical state protection question.

Detailed Technical Answer: It acts as an indestructible safety lock on stateful resources (RDS databases, primary S3 buckets). If any execution plan attempts to destroy the resource—whether via 'terraform destroy' or by removing the resource block from code—Terraform halts execution with a fatal error. To genuinely delete it, an engineer must first update code to `prevent_destroy = false`, apply that change, and then delete.

■■ **Spoken English Interview Soundbite:**

"prevent_destroy is a guardrail against catastrophic accidental deletion. When active, Terraform hard-fails any plan that would destroy the resource. Decommissioning requires a deliberate two-step code change to set the rule to false first."

QUESTION 23

How does 'ignore_changes' resolve configuration drift with AWS Auto Scaling or security tools?

Context & Why it's asked: Solves the classic 'Terraform fighting Cloud' dilemma.

Detailed Technical Answer: In production, external systems modify resources (e.g. AWS Auto Scaling scales `desired_capacity` from 2 to 10, or security scanners append compliance tags). Normally, Terraform detects this as drift and reverts them on every apply. `ignore_changes = [desired_capacity, tags]` instructs Terraform to manage the core resource while completely ignoring external modifications to those specific attributes.

■■ **Spoken English Interview Soundbite:**

"ignore_changes prevents Terraform from fighting cloud-native automation. We declare dynamic attributes like Auto Scaling desired_capacity or external security tags in the ignore_changes list so Terraform maintains the resource without reverting live operational updates."

SECTION 9: PRODUCTION CODE QUALITY, SECURITY & CI/CD GOVERNANCE

QUESTION 24

What does 'sensitive = true' achieve, and does it encrypt secrets inside terraform.tfstate?

Context & Why it's asked: Critical security question.

Detailed Technical Answer: Setting sensitive = true on variables and outputs instructs Terraform to redact the value from CLI outputs, terminal screens, and CI/CD log streams, displaying '(sensitive value)' instead. **Crucial Gotcha:** It does NOT encrypt the value in the state file! The secret is still stored in plain-text JSON in terraform.tfstate, which is why remote S3 encryption (KMS) and strict IAM access policies are mandatory.

■■ **Spoken English Interview Soundbite:**

"sensitive = true redacts credentials from console logs and CI/CD output to prevent credential leakage. However, it does not encrypt values in the state file, which reinforces why remote state encryption and strict S3 access controls are mandatory."

QUESTION 25

Compare 'terraform validate', 'TFLint', and 'tfsec / Checkov'. How are they automated in CI/CD?

Context & Why it's asked: Distinguishes top-tier DevOps candidates.

Detailed Technical Answer: terraform validate: Verifies basic HCL syntax and internal references without calling cloud APIs.

TFLint: A specialized linter that catches provider-specific errors (invalid EC2 instance types, deprecated syntax, naming conventions).

tfsec / Checkov: Static Application Security Testing (SAST) tools for IaC that scan for security vulnerabilities (open port 22 on 0.0.0.0/0, unencrypted S3 buckets, missing logging).

In CI/CD, these run as automated Pull Request quality gates before 'terraform plan'.

■■ **Spoken English Interview Soundbite:**

"In our CI/CD pipeline, we run layered quality gates: terraform validate for syntax integrity, TFLint for cloud-specific linting, and tfsec or Checkov to block security misconfigurations before generating execution plans."

QUESTION 26

Explain the end-to-end Enterprise Terraform CI/CD pipeline workflow.

Context & Why it's asked: The standard system design question for Cloud & DevOps engineers.

Detailed Technical Answer:

1. Developer creates a feature branch and opens a GitHub Pull Request.
2. GitHub Actions triggers automated checks: **terraform fmt -check**, **validate**, and **tfsec**.
3. Pipeline runs **terraform plan** and posts the execution plan diff directly as a comment on the PR.
4. Tech Lead reviews the plan diff and approves the PR.
5. On merge to main branch, the production pipeline runs **terraform apply -auto-approve** in an isolated runner.
6. Developers are never granted direct write permissions from local laptops.

■■ **Spoken English Interview Soundbite:**

"Our enterprise pipeline enforces GitOps governance: opening a PR triggers automated format, validation, and security scanning, followed by a plan whose diff is posted directly to the PR. Merging to main executes terraform apply through an automated runner, ensuring zero direct cloud writes from developer laptops."

SECTION 10: SCENARIO QUESTIONS FOR DELOITTE, EY & HCLTECH

QUESTION 27

SCENARIO: 'Our production website experienced a 5-minute outage after an EC2 AMI update. What caused it and how do you resolve it?'

Context & Why it's asked: Deloitte and EY production troubleshooting question.

Detailed Technical Answer: The outage occurred because Terraform's default replacement sequence is Destroy-then-Create (-/+). Updating an AMI cannot be applied in-place, so Terraform terminated the running EC2 instance before the replacement booted and passed health checks. We resolve this by adding **lifecycle { create_before_destroy = true }** to ensure the replacement instance is fully operational before the old instance is terminated.

■■ **Spoken English Interview Soundbite:**

"This outage occurred because of Terraform's default Destroy-then-Create behavior during AMI replacements. To achieve zero downtime, we implement lifecycle create_before_destroy = true so the replacement instance is healthy and taking traffic before the previous instance is decommissioned."

QUESTION 28

SCENARIO: 'How did you architect Terraform for your Real-Time Collaborative Spreadsheet project?'

Context & Why it's asked: Tailored specifically for Priyanshu Jaiswal to showcase full-stack cloud mastery.

Detailed Technical Answer: In the Collaborative Spreadsheet project, manual cloud configuration would lead to configuration drift and reproducibility issues. We architected modular Terraform configurations: a networking module (VPC, public/private subnets, security groups) and a compute module (EC2 instances running containerized Spring Boot services). We enforced lifecycle rules: **prevent_destroy** on the PostgreSQL database to safeguard sheet cell history, and **create_before_destroy** on API Gateway/EC2 to preserve live WebSocket STOMP connections during updates.

■■ Spoken English Interview Soundbite:

"In my Real-Time Collaborative Spreadsheet project, I automated our AWS infrastructure using modular Terraform. I built reusable modules for networking and compute, secured our PostgreSQL database against accidental deletion using prevent_destroy, and applied create_before_destroy so WebSocket connections never experience downtime during deployments."